TTA-REO Simulation

User Tutorial: Inputs, Outputs and Workflow

Companion to the paper

Tat Tvam Asi-Inspired Relational Entanglement Orchestration for Quantum Communication Networks

Swarup Roy
Department of Computer Science and Engineering, Tezpur University
swarup@tezu.ernet.in

Rupak Mukherjee
Department of Physics, Sikkim University
rmukherjee@cus.ac.in

October 2026. All inputs and outputs in this tutorial were produced by running the code in the package `TTA_REO_simulation_code.zip`.

Contents

1 What the code does **2**
1.1 Files in the package . . . 2
1.2 Workflow . . . 2

2 Installation **2**

3 Tutorial 1: the worked example **3**
3.1 Running it . . . 3
3.2 Reading the output . . . 4

4 Tutorial 2: calling the simulator from Python **4**
4.1 Inputs . . . 4
4.2 Output . . . 5

5 Tutorial 3: changing parameters **5**
5.1 Parameter reference . . . 6

6 Tutorial 4: reproducing the paper's experiments **6**
6.1 Running the experiments . . . 6
6.2 Files written . . . 7

7 Tutorial 5: understanding the output files **7**
7.1 Raw results . . . 7
7.2 Summary results . . . 7
7.3 Loading results into pandas . . . 8

8 Tutorial 6: figures and tables for the paper **8**

9 Extending the code **9**
9.1 Adding an experiment . . . 9
9.2 Adding a policy . . . 10

10 Troubleshooting **10**

11 Scope and limitations **10**

1 What the code does

The package simulates a six-node quantum network in which many entanglement requests compete for limited quantum memory. Requests arrive at random, each asking for a Bell pair or a GHZ state before a deadline. Five allocation policies decide which links to try to entangle and which pair goes to which request. The simulator records how many requests each policy completes, and the experiment scripts repeat this over 300 random seeds to produce every number, table and figure in the paper.

1.1 Files in the package

File	Purpose
`tta_sim.py`	The simulator. Contains all parameters (`PARAMS`), the request generator and `simulate()`.
`example_single_run.py`	Short worked example. Start here.
`sweep.py`	Runs the four experiments of the paper with 300 seeds each and computes the statistics.
`make_figures.py`	Builds the results figure and the LaTeX table rows from the summaries.
`results/*_raw.json`	Per-seed outcome of every policy in every setting.
`results/*_summary.json`	Means, confidence intervals and significance tests.
`requirements.txt`	Python packages needed.
`README.md`	Short reference version of this tutorial.

1.2 Workflow

Figure 1 shows how the parts fit together. You can stop at any stage: a single call to `simulate()` gives one result; `sweep.py` gives the full statistics; `make_figures.py` turns them into material for the paper.

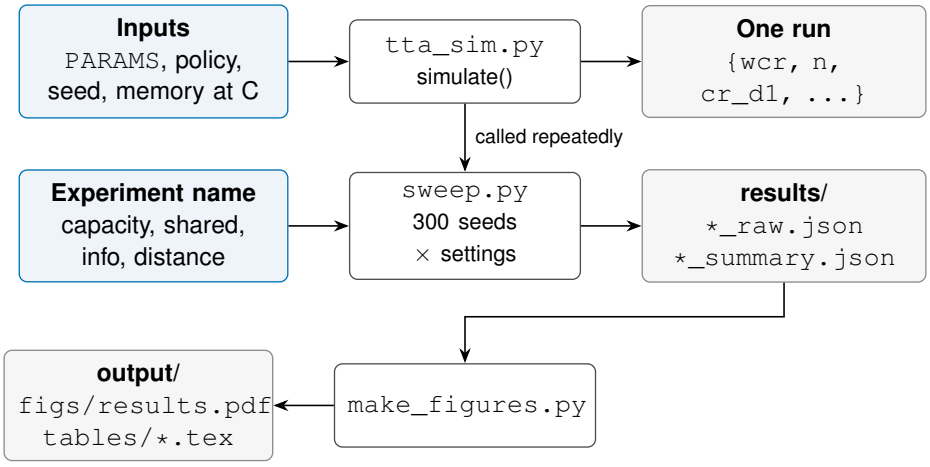


Figure 1: Workflow. Shaded blue boxes are inputs you choose; grey boxes are outputs written by the code.

2 Installation

You need Python 3.9 or later. The code was tested with Python 3.13, NumPy 2.5, SciPy 1.18 and Matplotlib 3.11. No GPU is required. Unzip the package, open a terminal in the folder `TTA-REO-simulation`, and run:

INPUT

```
python -m venv venv
source venv/bin/activate          # on Windows: venv\Scripts\activate
pip install -r requirements.txt
```

The virtual environment is optional but keeps the packages separate from your system Python. Pandas is needed only for the optional analysis in Section 7.3.

3 Tutorial 1: the worked example

3.1 Running it

INPUT

```
python example_single_run.py
```

The script, shown in full below, does three things: it prints the link quantities derived from the physical parameters; it runs every policy once on seed 1000 with three memory qubits at node C; and it repeats three of the policies with older controller information.

INPUT

```
"""
Small worked example: run every policy once on one seed, then show how to
change a parameter. Takes a few seconds.

    python example_single_run.py
"""
import copy

from tta_sim import PARAMS, POLICIES, simulate, make_workload, derived_table

SEED = 1000
CAP_C = 3          # memory qubits at the shared node C

# 1. Quantities derived from the physical parameters (Table 5 of the paper)
d = derived_table(PARAMS)
print("Link  length(km)  p_success  herald(slots)")
for e, v in d["links"].items():
    print(f"{e:4s}  {v['length_km']:9.0f}  {v['p']:9.3f}  {v['herald_slots
    ']:13d}")
print("Correction delay (slots):", d["correction_slots"], "\n")

# 2. One run per policy on the same workload (common random numbers)
wl = make_workload(SEED, PARAMS)
print(f"Seed {SEED}, C capacity {CAP_C}, {len(wl)} requests")
print("Policy             WCR    d1     d2     d3")
for pol in POLICIES:
    r = simulate(pol, SEED, CAP_C, PARAMS, wl)
    print(f"{pol:16s} {r['wcr']:.3f}  {r['cr_d1']:.2f}   {r['cr_d2']:.2f}
    {r['cr_d3']:.2f}")

# 3. Changing a parameter: make the controller's information 4 slots old
P = copy.deepcopy(PARAMS)
P["INFO_DELAY"] = 4
wl = make_workload(SEED, P)
print("\nSame seed with INFO_DELAY = 4 slots (0.4 ms):")
for pol in ("TTA-lite", "RTC-direct", "Fidelity-greedy"):
    r = simulate(pol, SEED, CAP_C, P, wl)
    print(f"{pol:16s} {r['wcr']:.3f}")
```

OUTPUT

```
Link  length(km)  p_success  herald(slots)
AC           20      0.358              1
CD           25      0.285              2
BE           15      0.451              1
```

```
EF           10      0.568              1
CF           30      0.226              2
BC           20      0.358              1
Correction delay (slots): {'d1': 2, 'd2': 1, 'd3': 0}

Seed 1000, C capacity 3, 205 requests
Policy             WCR    d1     d2     d3
EDF-greedy       0.598  0.23   0.93   0.64
Fidelity-greedy  0.545  0.38   0.81   0.42
TTA-lite         0.606  0.20   0.87   0.55
TTA-lite-noCP    0.586  0.20   0.89   0.50
RTC-direct       0.631  0.28   0.93   0.59

Same seed with INFO_DELAY = 4 slots (0.4 ms):
TTA-lite         0.354
RTC-direct       0.298
Fidelity-greedy  0.230
```

3.2 Reading the output

Link table. For each link the code derives the per-slot success probability from its length using fibre loss of 0.2 dB/km, and the heralding delay, which is the number of 0.1 ms slots a signal needs to cross the link. Longer links succeed less often and take longer to confirm.

Correction delay. After the swap at C (request type d1) or the fusion at E (d2), the measurement result must reach the end nodes before the request counts as complete. This takes 2 slots for d1 and 1 slot for d2; d3 needs no operation.

Policy table. `WCR` is the weighted completion ratio: the share of request weight that was completed by its deadline (1.0 would mean every request was served). The columns `d1`, `d2`, `d3` give the fraction of each request type completed. All five policies saw exactly the same 205 requests and the same random outcomes, so differences come from the policies alone.

Older information. With the controller's information four slots old, every policy completes far fewer requests, because pairs sit unused while their reports are in transit and deadlines pass.

Note. One seed is one random sample. On this seed RTC-direct happens to score highest at $\Delta = 1$ and lower at $\Delta = 4$; averaged over 300 seeds the ordering is different (Section 6). Never draw conclusions from a single seed.

4 Tutorial 2: calling the simulator from Python

4.1 Inputs

The three functions you will use are listed below.

Function and arguments	Meaning
`make_workload(seed, P)`	Generates the list of requests for one seed. Returns a list of dictionaries (see below).
`simulate(policy, seed, cap_c, P, workload)`	Runs one simulation and returns the result dictionary.
`derived_table(P)`	Returns success probabilities, heralding delays and correction delays implied by `P`.

Argument	Type	Allowed values
`policy`	string	`"EDF-greedy"`, `"Fidelity-greedy"`, `"TTA-lite"`, `"TTA-lite-noCP"`, `"RTC-direct"`
`seed`	integer	Any integer. The paper uses 1000–1299; 0–14 were pilot seeds.
`cap_c`	integer	Memory qubits at node C, 1 or more. The paper uses 4, 3, 2, 1.
`P`	dict	Parameter dictionary, normally `PARAMS` or a modified copy.
`workload`	list	Output of `make_workload(seed, P)`. If omitted it is generated.

Pass the same `workload` to every policy you want to compare, so that they face identical requests. Each request in the workload looks like this (first three requests of seed 1000):

OUTPUT

```
[{'rid': 0, 'kind': 'd1', 'arrival': 0, 'deadline': 13, 'w': 3, 'fmin': 0.8490020314561623}, {'
    ↪ rid': 1, 'kind': 'd2', 'arrival': 1, 'deadline': 14, 'w': 1, 'fmin':
    ↪ 0.8093066935737938}, {'rid': 2, 'kind': 'd1', 'arrival': 4, 'deadline': 20, 'w': 3, '
    ↪ fmin': 0.7006007475241958}]
```

Here `kind` is the request type, `arrival` and `deadline` are in slots, `w` is the priority weight (1–3) and `fmin` is the fidelity threshold.

4.2 Output

`simulate()` returns a dictionary with five entries:

Key	Meaning
`wcr`	Weighted completion ratio (0 to 1)
`n`	Number of requests that arrived
`cr_d1`	Fraction of A–D Bell requests completed
`cr_d2`	Fraction of B–E–F GHZ requests completed
`cr_d3`	Fraction of C–F Bell requests completed

5 Tutorial 3: changing parameters

All parameters live in the dictionary `PARAMS` at the top of `tta_sim.py`. Rather than editing the file, make a copy in your own script and change what you need.

INPUT

```
import copy
from tta_sim import PARAMS, simulate, make_workload

P = copy.deepcopy(PARAMS)              # never edit PARAMS itself
P["INFO_DELAY"] = 2                    # controller information 2 slots old
P["EDGES"]["CF"] = (40.0, 0.92)        # CF link 40 km, initial quality 0.92
P["ARRIVAL_RATE"] = 0.5                # heavier load

wl = make_workload(1000, P)            # requests for seed 1000
result = simulate("TTA-lite", 1000, 3, P, wl)
print(result)
```

OUTPUT

```
{'wcr': 0.42105263157894735, 'n': 245, 'cr_d1': 0.16470588235294117, 'cr_d2':
    ↪ 0.6842105263157895, 'cr_d3': 0.27380952380952384}
```

With a longer CF link, older information and more traffic, TTA-lite completes 42% of request weight, and only 27% of the C–F requests that use the lengthened link.

5.1 Parameter reference

Key	Meaning	Paper value
`EDGES`	Each link: (length in km, initial quality)	AC (20, .95), CD (25, .95), BE (15, .93), EF (10, .97), CF (30, .92), BC (20, .94)
`P_MAX`	Success probability per slot at zero length	0.90
`ALPHA_DB_PER_KM`	Fibre attenuation	0.2
`TAU_SLOT_MS`	Length of one slot (ms)	0.1
`C_FIBRE_KM_PER_MS`	Signal speed in fibre	200
`MEM_DEFAULT`	Memory qubits at every node except C	4
`TAU_COH`	Quality decay constant (slots)	100
`LIFETIME`	Age at which a pair is discarded (slots)	20
`P_SWAP, P_FUSE`	Success of swap at C and fusion at E	0.90, 0.85
`LOSS`	Quality factor applied by a swap or fusion	0.98
`ARRIVAL_RATE`	Mean requests per slot (Poisson)	0.4
`HORIZON`	Slots during which requests arrive	500
`DEADLINE`	Range of relative deadlines (slots)	(8, 25)
`WEIGHTS`	Possible priority weights	(1, 2, 3)
`FMIN`	Fidelity threshold range per request type	see `tta_sim.py`
`STALE_EPS`	Age penalty used by TTA-lite	0.05
`INFO_DELAY`	Delay before the controller learns an outcome (slots)	1
`GEN_RANK`	`"policy"`: each policy orders generation; `"shared"`: one random order for all	`"policy"`
`FEASIBILITY_FILTER`	Ignore requests that cannot fit in memory	True

Because heralding and correction delays are rounded up to whole slots, small changes to a link length may not change the delay. Use `derived_table(P)` to check what your parameters imply before running long experiments.

6 Tutorial 4: reproducing the paper's experiments

6.1 Running the experiments

INPUT

```
python sweep.py                 # all four experiments (about 6 minutes on 2
    CPU cores)
python sweep.py capacity        # or one experiment at a time
```

Argument	Exp.	What varies	Fixed settings
`capacity`	A	Memory at C: 4, 3, 2, 1	$\Delta = 1$, policy-ranked generation
`shared`	B	Memory at C: 4, 3, 2, 1	$\Delta = 1$, one shared generation order
`info`	C	Report delay Δ: 0, 1, 2, 4, 8 slots	C = 3
`distance`	D	All link lengths $\times$ 0.5, 1, 1.5, 2	C = 3, $\Delta = 1$

Each setting is run for every policy on seeds 1000–1299, using all available CPU cores. Console output for `python sweep.py capacity` (abbreviated):

OUTPUT

```
cap=4: EDF-greedy=0.588  Fidelity-greedy=0.540  TTA-lite=0.622  TTA-lite-noCP=0.601  RTC-direct
    ↪ =0.596
cap=3: EDF-greedy=0.505  Fidelity-greedy=0.462  TTA-lite=0.557  TTA-lite-noCP=0.532  RTC-direct
    ↪ =0.518
cap=2: EDF-greedy=0.410  Fidelity-greedy=0.374  TTA-lite=0.488  TTA-lite-noCP=0.439  RTC-direct
    ↪ =0.422
cap=1: EDF-greedy=0.449  Fidelity-greedy=0.436  TTA-lite=0.462  TTA-lite-noCP=0.459  RTC-direct
    ↪ =0.453
cap=4 TTA-lite vs Fidelity-greedy: +15.3% W/L/T 294/6/0 p_holm=4.4e-49
```

```
cap=4 TTA-lite vs EDF-greedy: +5.8% W/L/T 246/50/4 p_holm=1e-32
cap=4 TTA-lite vs TTA-lite-noCP: +3.6% W/L/T 237/57/6 p_holm=1.4e-26
cap=4 RTC-direct vs Fidelity-greedy: +10.4% W/L/T 270/28/2 p_holm=2.3e-41
cap=4 RTC-direct vs EDF-greedy: +1.3% W/L/T 165/126/9 p_holm=0.0028
cap=4 RTC-direct vs TTA-lite: -4.3% W/L/T 77/216/7 p_holm=8.1e-23
...
```

The first block gives the mean WCR of every policy at each memory size. The second block lists the pairwise comparisons: relative difference in percent, the number of seeds on which the first policy won, lost or tied, and the Holm-adjusted p-value from a paired Wilcoxon test.

6.2 Files written

Each experiment writes two files to `results/`, replacing the shipped ones:

- `<experiment>_raw.json`: the result dictionary of every policy, for every seed and every setting;
- `<experiment>_summary.json`: means, confidence intervals and tests.

The simulation is deterministic. Rerunning `sweep.py capacity` on a fresh copy reproduced the shipped per-seed results exactly.

7 Tutorial 5: understanding the output files

7.1 Raw results

The raw file is nested as `results` → setting → seed → policy. For example, TTA-lite at memory size 2 on seed 1000:

OUTPUT


```
{
 "results": {
  "2": {
   "1000": {
    "TTA-lite": {
     "wcr": 0.5681818181818182,
     "n": 205,
     "cr_d1": 0.14754098360655737,
     "cr_d2": 0.8714285714285714,
     "cr_d3": 0.4594594594594595
    }
   }
  }
 }
}
```


The file also contains a `meta` block recording the experiment name, the varied parameter, its values, the seeds, the policies, the parameter settings and the derived link quantities.

7.2 Summary results

`per_value` gives, for each setting and policy, the mean WCR, its 95% bootstrap confidence interval and the mean completion rate of each request type:

OUTPUT


```
"per_value": { "2": { "TTA-lite": {
    "mean": 0.4876715208996182,
    "ci95": [0.48308262318800205, 0.49223606931840774],
    "per_type": { "d1": 0.1072, "d2": 0.8065, "d3": 0.4442 } } } }
```


`comparisons` lists every pairwise test. TTA-lite against Fidelity-greedy at memory size 2:

OUTPUT


```
{ "value": 2, "a": "TTA-lite", "b": "Fidelity-greedy",
  "mean_a": 0.4877, "mean_b": 0.3736,
```

```
  "diff": 0.1141, "diff_ci95": [0.1091, 0.1190],
  "rel_pct": 30.54,
  "wins": 299, "losses": 1, "ties": 0,
  "p": 6.14e-51, "p_holm": 1.47e-49 }
```

Field	Meaning
`value`	The setting (memory size, delay or length scale)
`a, b`	The two policies compared
`mean_a, mean_b`	Mean WCR of each over 300 seeds
`diff, diff_ci95`	Mean paired difference $a - b$ and its 95% bootstrap interval
`rel_pct`	Relative difference $100\,(\text{mean_a}/\text{mean_b} - 1)$, the number quoted in the paper
`wins, losses, ties`	Number of seeds on which a beat, lost to or tied with b
`p`	Two-sided Wilcoxon signed-rank p-value
`p_holm`	p-value after Holm correction across all tests of the experiment (reported in the paper)

7.3 Loading results into pandas

For your own analysis, the raw file converts easily into a table:

INPUT

```
import json
import pandas as pd

raw = json.load(open("results/capacity_raw.json"))
rows = []
for cap, seeds in raw["results"].items():
    for seed, pols in seeds.items():
        for pol, r in pols.items():
            rows.append(dict(cap=int(cap), seed=int(seed), policy=pol, **r))
df = pd.DataFrame(rows)
print(df.head(3).to_string(index=False))
print()
print(df.pivot_table(index="cap", columns="policy", values="wcr").round(3).
    to_string())
```

OUTPUT

```
 cap  seed          policy      wcr   n    cr_d1    cr_d2    cr_d3
   4  1000      EDF-greedy 0.628788 205 0.245902 0.914286 0.635135
   4  1000 Fidelity-greedy 0.580808 205 0.426230 0.814286 0.486486
   4  1000        TTA-lite 0.676768 205 0.311475 0.857143 0.662162

policy  EDF-greedy  Fidelity-greedy  RTC-direct  TTA-lite  TTA-lite-noCP
cap
1            0.449            0.436       0.453     0.462          0.459
2            0.410            0.374       0.422     0.488          0.439
3            0.505            0.462       0.518     0.557          0.532
4            0.588            0.540       0.596     0.622          0.601
```

8 Tutorial 6: figures and tables for the paper

INPUT

```
python make_figures.py                    # writes into ./output
python make_figures.py --out ../paper     # writes into ../paper/figs and ../
    paper/tables
```

OUTPUT

```
wrote ./output/figs and ./output/tables
```

This reads the four summary files and writes:

- `figs/results.pdf` (and a `.png` preview): the four-panel results figure of the paper, reproduced in Figure 2;
- `tables/capacity.tex`, `shared.tex`, `info.tex`, `distance.tex`: the body rows of the four results tables.

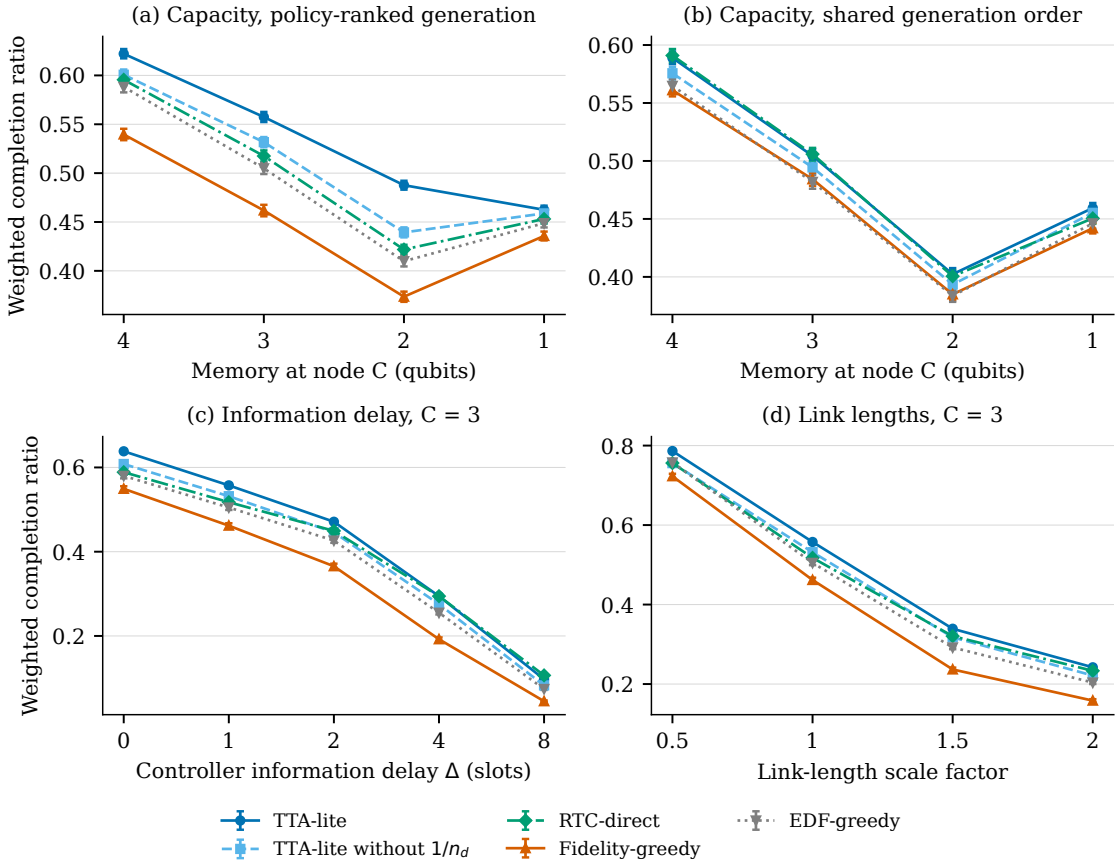


Figure 2: Output of `make_figures.py`: weighted completion ratio with 95% confidence intervals for experiments A–D.

One line of `tables/capacity.tex` (memory size 2) reads:

OUTPUT

```
2 & 0.488 & 0.439 & 0.422 & 0.374 & 0.410 & $+30.5$ ($1.5\times 10^{-49}$) & $+19.0$ ($1.6\
    ↪ times 10^{-48}$) & $+11.0$ ($7.2\times 10^{-44}$) & $-13.5$ ($9.3\times 10^{-48}$) \\
```

The columns are: setting; mean WCR of TTA-lite, TTA-lite without $1/n_d$, RTC-direct, Fidelity-greedy and EDF-greedy; then the relative difference (with Holm-adjusted p) of TTA-lite against Fidelity-greedy, EDF-greedy and its ablation, and of RTC-direct against TTA-lite. The paper's LaTeX source reads these files directly, so after re-running the experiments you only need to copy `figs/` and `tables/` into the Overleaf project.

9 Extending the code

9.1 Adding an experiment

Experiments are defined in the `EXPERIMENTS` dictionary in `sweep.py`. To sweep the arrival rate, for example, add an entry and a matching branch in `params_for()`:

INPUT

```
EXPERIMENTS["load"] = dict(var="ARRIVAL_RATE", values=[0.2, 0.4, 0.6, 0.8],
```

```
                                base=dict(GEN_RANK="policy", INFO_DELAY=1), cap
    =3)

# inside params_for(exp, value):
    if EXPERIMENTS[exp]["var"] == "ARRIVAL_RATE":
        P["ARRIVAL_RATE"] = value
```

Then run `python sweep.py load`. The results are written to `results/`, in the files `load_raw.json` and `load_summary.json`.

9.2 Adding a policy

Policies are scores in `match_score()` in `tta_sim.py`; a higher score is preferred. Add the name to `POLICIES` and a branch to `match_score()`. The score must be a tuple, since tuples are compared element by element.

INPUT

```
POLICIES = (..., "Weight-greedy")

# inside match_score(...):
    if policy == "Weight-greedy":
        return (req.w, pair_q)          # highest weight first, then best
    pair
```

10 Troubleshooting

Symptom	Remedy
`ModuleNotFoundError: numpy` (or scipy, matplotlib)	Activate the environment and run `pip install -r requirements.txt`.
`unknown experiment(s)`	Use one of `capacity`, `shared`, `info`, `distance`, or add your own (Section 9.1).
`sweep.py` is slow	It uses all CPU cores; on one core allow about 12 minutes. For quick tests reduce `SEEDS` in `sweep.py`, for example to `range(1000, 1030)`.
Shipped results were overwritten	Unzip the package again to restore them, or rerun: results are deterministic.
Numbers differ slightly from the paper	Check that `PARAMS` and `SEEDS` are unchanged. Seeds are fixed, so results should match; very different NumPy or SciPy versions can in rare cases change the last digits. Record the versions you used.
`make_figures.py` fails with a missing file	It needs all four summaries in `results/`. Run `python sweep.py` first.

11 Scope and limitations

The simulator is an abstract discrete-time model built to compare allocation rules, not a physical-layer simulator. Quality is a single number standing in for fidelity, and density matrices are not tracked. Operation success rates are fixed, the report delay is the same for every node, and only the six-node network of the paper is modelled. For physical-layer studies, tools such as NetSquid or SeQUeNCe are more appropriate.

If you use the code, please cite the paper. Questions: Swarup Roy (`swarup@tezu.ernet.in`) or Rupak Mukherjee (`rmukherjee@cus.ac.in`).